Agentic MLOps Platform
for LLM Application Lifecycle Management

Sinh viên	Dương Công Thuyết
MSSV	20225932

1. Vấn Đề Thực Tế (Problem Statement)

Khi các nhóm AI đưa LLM application vào production, một câu hỏi cực kỳ thực tế xuất hiện:

> ***"Khi tôi thay đổi prompt, swap model, hoặc update tool — làm sao tôi biết hệ thống có bị regression không?"***

Trong phát triển phần mềm truyền thống, CI/CD pipeline đã giải quyết vấn đề này với unit test và integration test. Nhưng với LLM-based agent systems, pipeline truyền thống không phù hợp vì:

- Output của LLM mang tính xác suất, không deterministic — không thể assert bằng assertEqual()
- Prompt là "code" nhưng không được version-control và test một cách hệ thống
- Agent có nhiều tool calls, mỗi thay đổi nhỏ có thể lan rộng ảnh hưởng theo chuỗi
- Không có chuẩn mực (standard) nào cho việc evaluate và deploy LLM agent trong production

Hậu quả thực tế: các công ty hiện tại đang patch bằng tay — test thủ công, rollback thủ công, không có visibility vào quality drift theo thời gian. Đây là production gap nghiêm trọng khi LLM app ngày càng được deploy rộng rãi.

2. Câu Hỏi Nghiên Cứu (Research Questions)

1. **RQ1 (Core):** Làm thế nào để thiết kế một agentic pipeline có khả năng tự động evaluate quality của LLM application sau mỗi thay đổi về prompt, model, hoặc tool configuration?
2. **RQ2 (Eval Design):** Metric nào phù hợp để đánh giá quality regression của LLM agent (latency, accuracy, cost, tool-call correctness) và làm thế nào để kết hợp chúng thành một decision signal duy nhất?
3. **RQ3 (Architecture):** Model Context Protocol (MCP) server có thể đóng vai trò gì trong việc chuẩn hóa giao tiếp giữa orchestration agent và các MLOps tools (deployment, monitoring, version store)?
4. **RQ4 (Validation):** Platform đề xuất có giảm thiệt được thời gian phát hiện regression và thời gian rollback so với quy trình thủ công không, và ở mức độ nào?

3. Đóng Góp Khoa Học (Research Contributions)

C1 — Framework mới cho LLM Application Lifecycle

Đề xuất kiến trúc AgentOps Pipeline: một framework multi-agent sử dụng LangGraph để tự động hóa toàn bộ vòng đời của LLM application từ change detection → evaluation → promotion/rollback decision. Đây là contribution mang tính architectural, khác biệt với các công trình RAG hay chatbot thông thường.

C2 — Composite Eval Metric cho Agent Systems

Thiết kế và validate một bộ metric đánh giá chất lượng agent gồm nhiều chiều: task completion rate, tool-call accuracy, latency, cost efficiency, và output quality (LLM-as-judge). Đề xuất cách kết hợp thành Quality Score duy nhất phục vụ automated decision-making.

C3 — MCP-based Tool Abstraction Layer cho MLOps

Triển khai và đánh giá việc dùng Model Context Protocol (MCP) server làm abstraction layer chuẩn hóa giữa orchestration agent và MLOps infrastructure (AWS S3, CloudWatch, deployment service). Đây là ứng dụng MCP trong MLOps context — một hướng còn rất ít nghiên cứu.

C4 — Case Study Validation

Validate toàn bộ platform trên một LLM application thực tế (Vietnamese document translation agent) với ít nhất 50 test cases được thiết kế đặc biệt cho bài toán này, đo lường được improvement rõ ràng về time-to-detect-regression và rollback latency.

4. Kiến Trúc Hệ Thống Đề Xuất

Layer	Component	Technology	Vai trò
Trigger	Change Detector	GitHub Webhook + AWS Lambda	Phát hiện thay đổi prompt/code/config
Orchestration	Eval Orchestrator Agent	LangGraph + Python	Điều phối toàn bộ eval pipeline
Evaluation	Eval Runner Agent	LangGraph + Gemini Flash	Chạy test suite, tính Quality Score
Comparison	Version Comparator Agent	LangGraph	So sánh v_new vs v_current, phát hiện regression

Layer	Component	Technology	Vai trò
Decision	Promotion Decision Agent	LangGraph	Promote / rollback / escalate dựa trên threshold
Tool Layer	MCP Server: Storage	FastAPI + AWS S3	Quản lý prompt versions, eval datasets, results
Tool Layer	MCP Server: Monitor	FastAPI + AWS CloudWatch	Đọc/ghi metrics sau deployment
Tool Layer	MCP Server: Deploy	FastAPI + AWS Lambda/EC2	Trigger deployment hoặc rollback
Storage	Version Store	AWS S3 + DynamoDB	Lưu trữ toàn bộ versions và metadata
Observability	Quality Dashboard	Next.js + React	Visualize metric drift theo version và thời gian

5. Timeline Dự Kiến (4 Tháng)

Giai đoạn	Thời gian	Milestone	Deliverable
Phase 0: Foundation	Tuần 1–2	Setup AWS infra, build target app (translation agent)	EC2 t3.micro + S3 + target app chạy được
Phase 1: MCP Servers	Tuần 3–5	Build 3 MCP servers: Storage, Monitor, Deploy	3 MCP servers hoạt động với unit tests
Phase 2: Eval Pipeline	Tuần 6–9	Build Eval Orchestrator + Eval Runner Agent bằng LangGraph	Pipeline tự động chạy eval khi có trigger
Phase 3: Decision Layer	Tuần 10–12	Build Comparator + Decision Agent, define Quality Score	Full automated promote/rollback hoạt động
Phase 4: Dashboard	Tuần 13–14	Build Next.js dashboard, polish integration	Dashboard live với metric drift visualization
Phase 5: Validation	Tuần 15–17	Thiết kế 50 test cases, đo kết quả, so sánh baseline	Kết quả đo lường, phân tích, bảng so sánh
Phase 6: Writing	Tuần 18–20	Viết báo cáo, chuẩn bị bảo vệ	Báo cáo hoàn chỉnh + slide bảo vệ

6. Tech Stack & AWS Services

Service	Mục đích
AWS S3	Lưu prompt versions, eval datasets, results
AWS Lambda	Trigger eval pipeline từ GitHub webhook
AWS DynamoDB	Version metadata, run history
AWS CloudWatch	Logs + metrics sau deployment
AWS EC2 t3.medium	Host orchestrator agent + MCP servers
Gemini Flash API	LLM cho eval agents + target app
GitHub	Version control + webhook trigger

7. Đánh Giá Rủi Ro & Giải Pháp

Rủi ro	Mức độ	Giải pháp
Chưa có real production app để test trên	Trung bình	Dùng translation agent từ internship tại Maxflow làm target app — tiết kiệm 2–3 tuần
Định nghĩa Quality Score metric không rõ ràng	Cao	Bắt đầu từ Phase 1 đã research metric, tham khảo RAGAS, DeepEval frameworks
AWS EC2 hết free tier sau 12 tháng	Thấp	Timeline 4 tháng nằm trong free tier; nếu cần extend thì chi phí ~$8/tháng
MCP server integration phức tạp hơn dự kiến	Trung bình	Build MCP servers theo thứ tự độ phức tạp tăng dần: Storage → Monitor → Deploy
LangGraph orchestration khó debug	Trung bình	LangGraph có built-in LangSmith tracing; bật từ đầu để dễ debug